

Machine Learning for Natural Language Understanding

Prof. Achim Rettinger



Transformers

Lecture 6 – Encoder

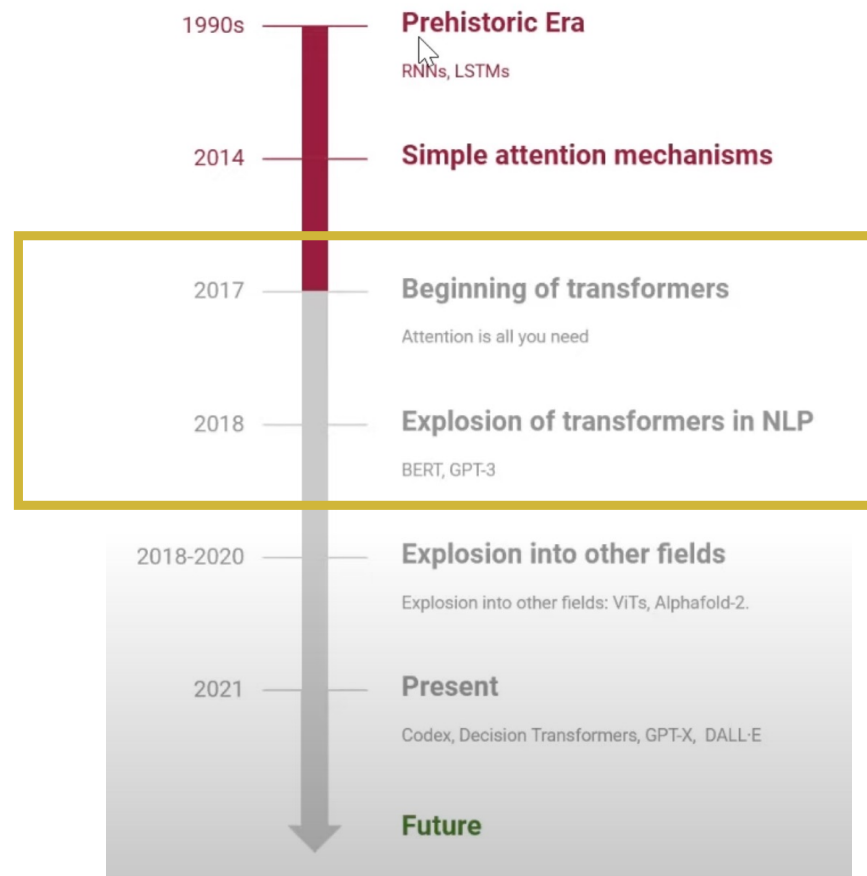
Sources Lecture

- Transformers
 - <https://youtu.be/ptuGIIU5SQQ?t=128> (Intro)
 - <https://youtu.be/ptuGIIU5SQQ?t=275> (Issues with RNNs)
 - <https://youtu.be/ptuGIIU5SQQ?t=826> (intra-sentence attention)
 - <https://youtu.be/ptuGIIU5SQQ?t=1442> (barriers and solutions of self-attention)
 - <https://youtu.be/ptuGIIU5SQQ?t=2191> (Transformer model)
 - <https://youtu.be/tlvKXrEDMhk?si=06qwofRpcEZqQ19N> (Keys, Values, Queries)
 - <https://youtu.be/ptuGIIU5SQQ?t=2781> (Tricks for Training)
 - <https://youtu.be/ptuGIIU5SQQ?t=3489> (Decoder cross-attention)
 - <https://youtu.be/ptuGIIU5SQQ?t=3902> (empirical results)
- <https://web.stanford.edu/class/cs224n/>

Flipped Classroom Questions

- Which context does a W2V embedding capture compared to an RNN embedding?
- What is self-attention?
- What context does a Bert embedding capture?
- What does an attention weight in Bert do?

Historic timeline – Focus of lecture today

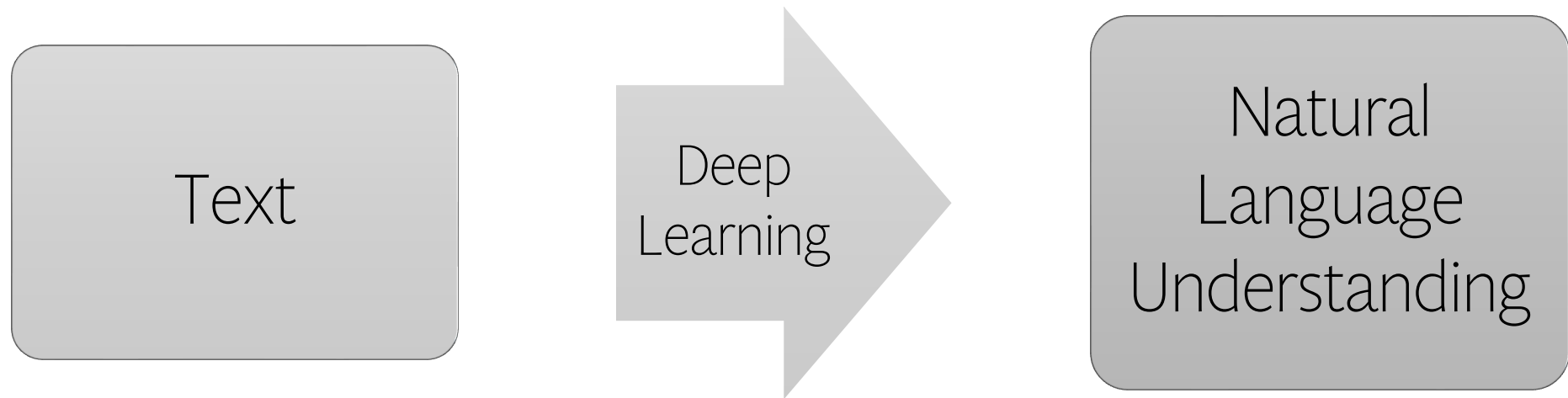


<https://youtu.be/P127jhj-8-Y>

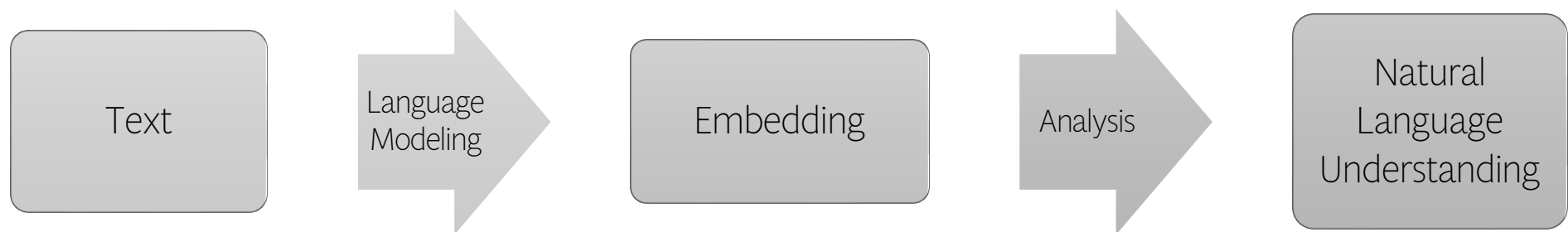
Prelude: Representation Learning

Chapter 6.1

Deep Learning Pipeline

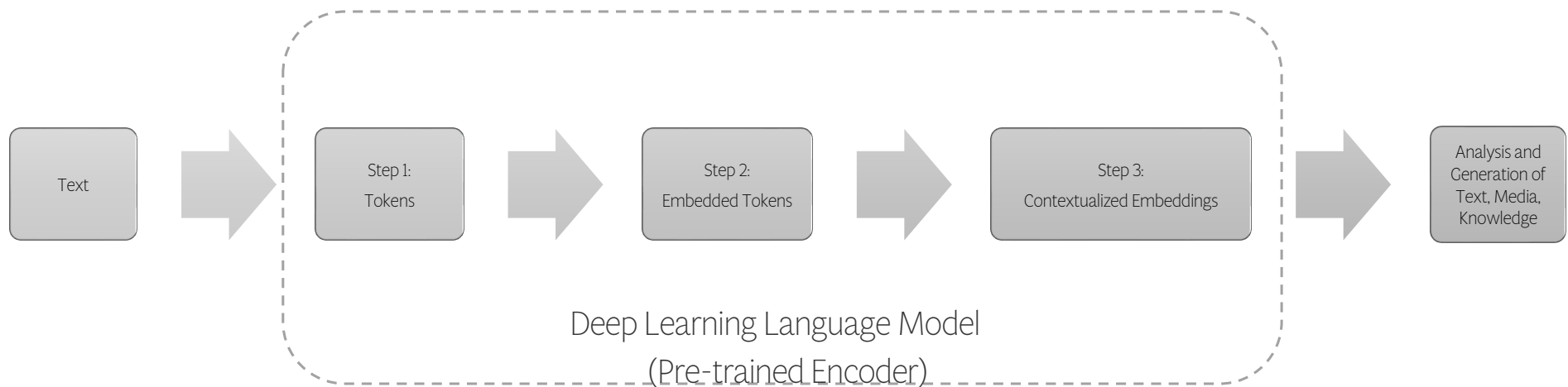


Representation Learning



- „Representation Learning“ uses Machine Learning to learn a function that transform the raw text input to feature vectors.
- The generated feature vectors are also called „embeddings“.

Steps of Language Modeling



Step1: Subword Tokenization

- Goal: Find a collection (= “vocabulary”) of atomic units (= “tokens”)
- Idea: Define statistically frequent character sequences as tokens
 - Multi-word phrases can be atomic tokens if they occur frequently
 - Handle rare words, abbreviations and acronyms
 - Same tokenization algorithm can be applied to any language

Step2: Token Embeddings with Word2Vec

- Goal: Capture the distributional semantics of each token
- Represent every token in the vocabulary with a vector
- Analyze a large corpus, find all occurrences of a specific token, analyze the surrounding words, capture those cooccurrences implicitly in the vector.

„Let's represent this sentence.“

Step 3: Customizing Embeddings given Context

- Goal: Transform vocabulary-based embeddings to contextualized embeddings depending on the surrounding words.
- Each occurrence of a token in the corpus gets an individual embedding.
- Capture syntactic information and semantic roles on the sentence level



Aspects of Transformers

Chapter 6.2 - <https://arxiv.org/pdf/2207.09238.pdf>

Input Tokenization

Real-world Setting	Data Representation	Learning Paradigm	Task
	• Feature vector • Sequence tokenization • Character-level • Word-level • Subword + Special tokens • Graph •	• (Self-)Supervised • Unsupervised • Reinforcement • ...	• Regression • Classification • Sequence Modelling • Seq-to-Seq • Clustering • ...
Model Instantiation	Model Representation	Optimization Technique	Objective Function
	Transformer	Stochastic Gradient Descent	Cross-Entropy Loss

Transformer Architectures

Real-world Setting	Data Representation	Learning Paradigm	Task
	• Feature vector • Sequence tokenization • Character-level • Word-level • Subword + Special tokens • Graph •	• (Self-)Supervised • Unsupervised • Reinforcement • ...	• Regression • Classification • Sequence Modelling • Seq-to-Seq • Clustering • ...
Model Instantiation	Model Representation	Optimization Technique	Objective Function
	Transformer • Encoder only • Encoder-Decoder • Decoder only (see next lecture)	Stochastic Gradient Descent	Cross-Entropy Loss

A cartoon illustration of Bert the Muppet. He is a yellow, bird-like character with a large orange beak and a wide, happy smile. He is wearing a blue suit jacket over a white shirt and a black tie. In his right hand, he holds a blue walkie-talkie to his mouth, and in his left hand, he holds a white notepad. The background is a blue pattern of small white circles. A semi-transparent yellow banner is overlaid across the middle of the image, containing the title and chapter information.

Bert Transformer

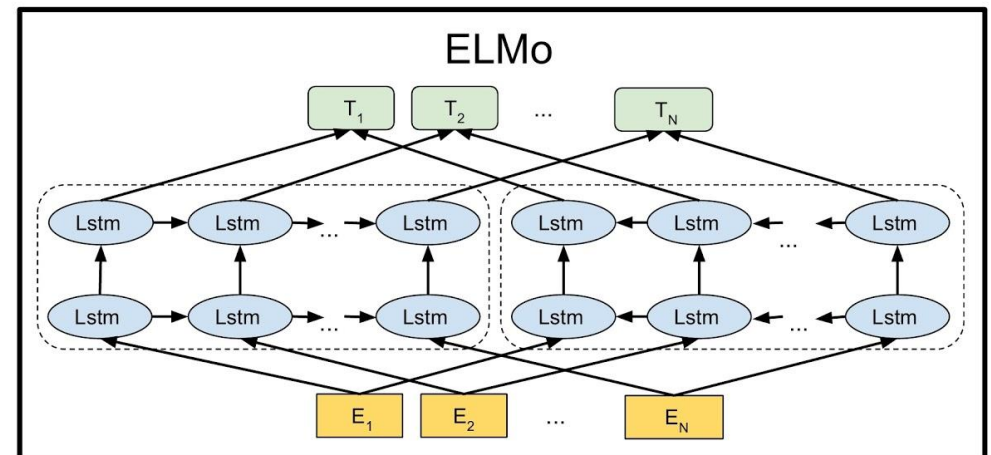
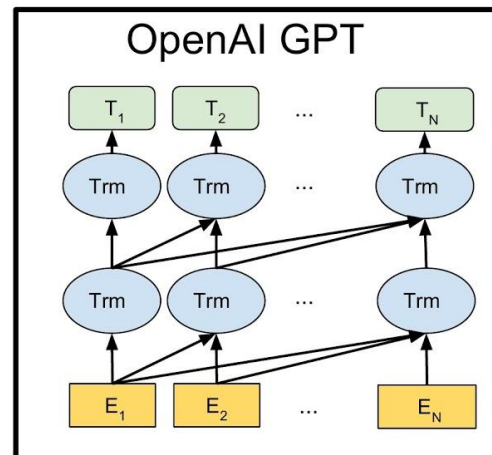
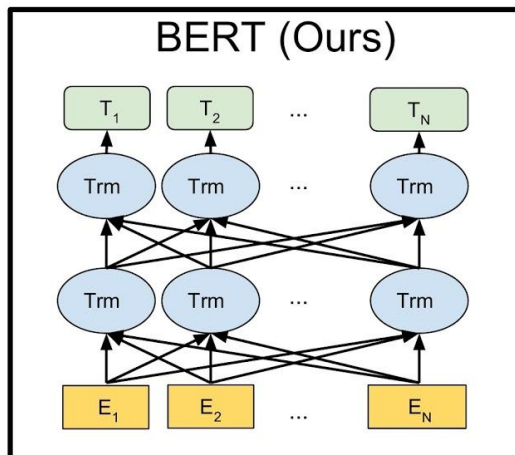
Chapter 6.3

<https://dl.acm.org/doi/10.5555/3295222.3295349>,

<https://towardsml.com/2019/09/17/bert-explained-a-complete-guide-with-theory-and-tutorial/>

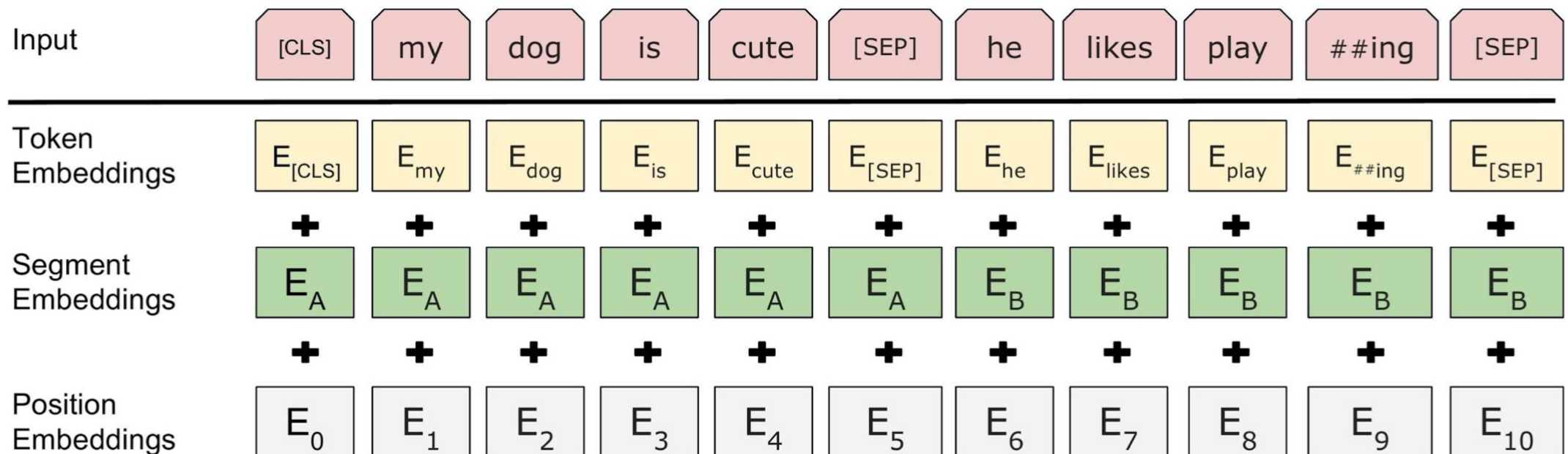
Representing sentences as vectors

- Naive: Combining W2V-vectors of all words in sentence (e.g., sum)
- LSTM / GPT: internal representation after reading the last word
- Bi-LSTMs (ELMo): reading sentence in both directions
- Transformers: pairwise comparison of all words in parallel (BERT)



Step 0: Bert Input

- Pairwise tokens is permutation invariant (changing the order does not change the learned representation)
- => Position Representation is needed



Bert – Encoder/Decoder

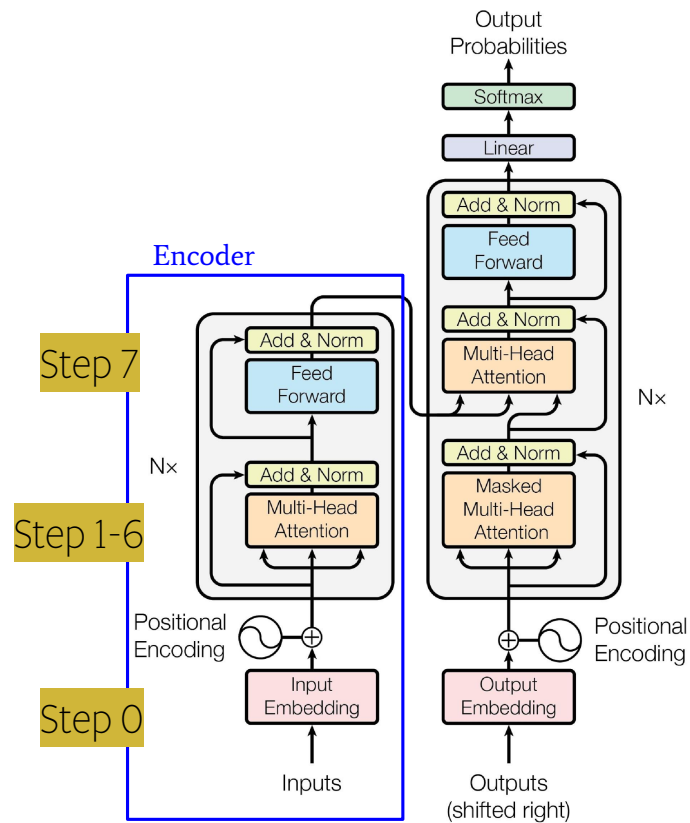
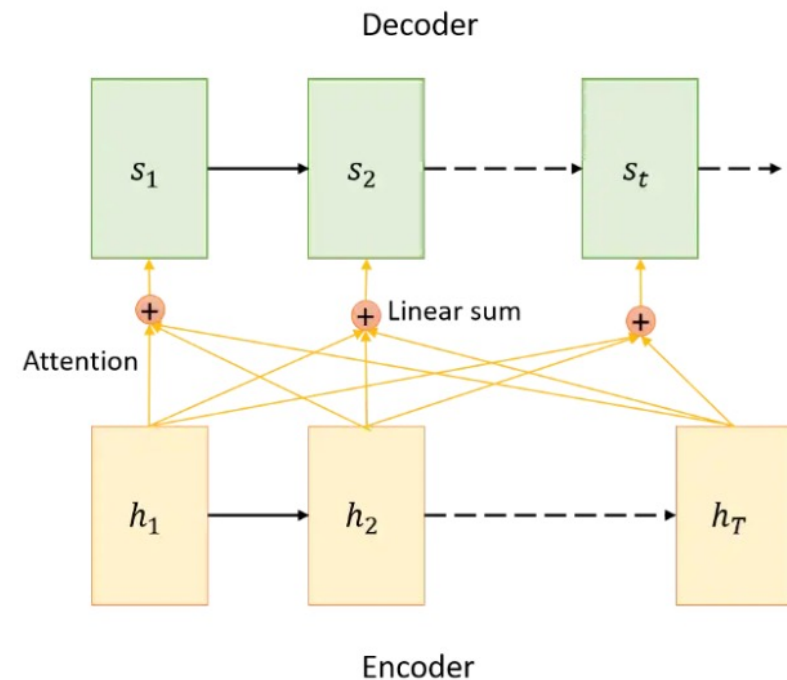
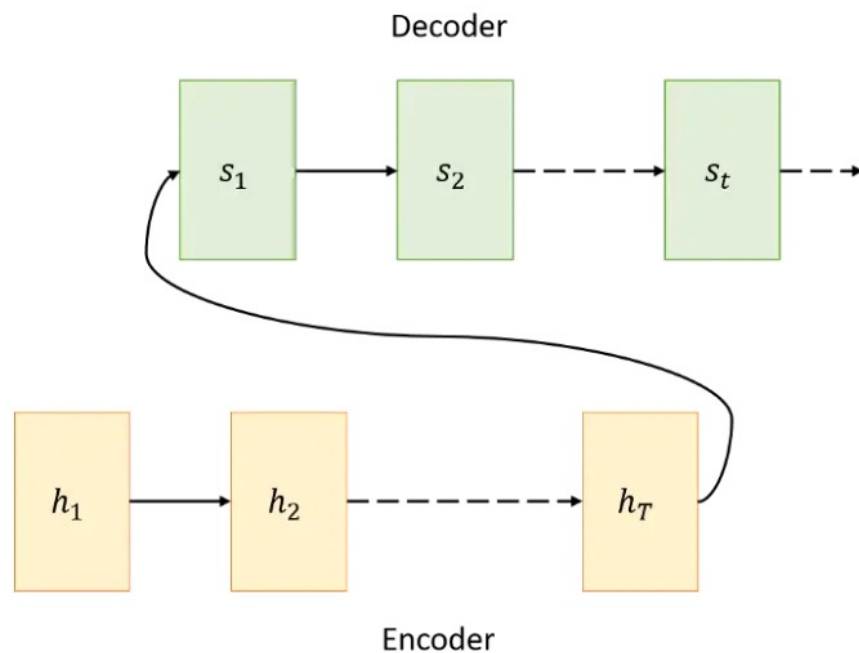


Figure 1: The Transformer - model architecture.

“Attention Is All You Need” (Vaswani et al., 2017)

Recap: Recurrent vs Attention based encoder-decoder



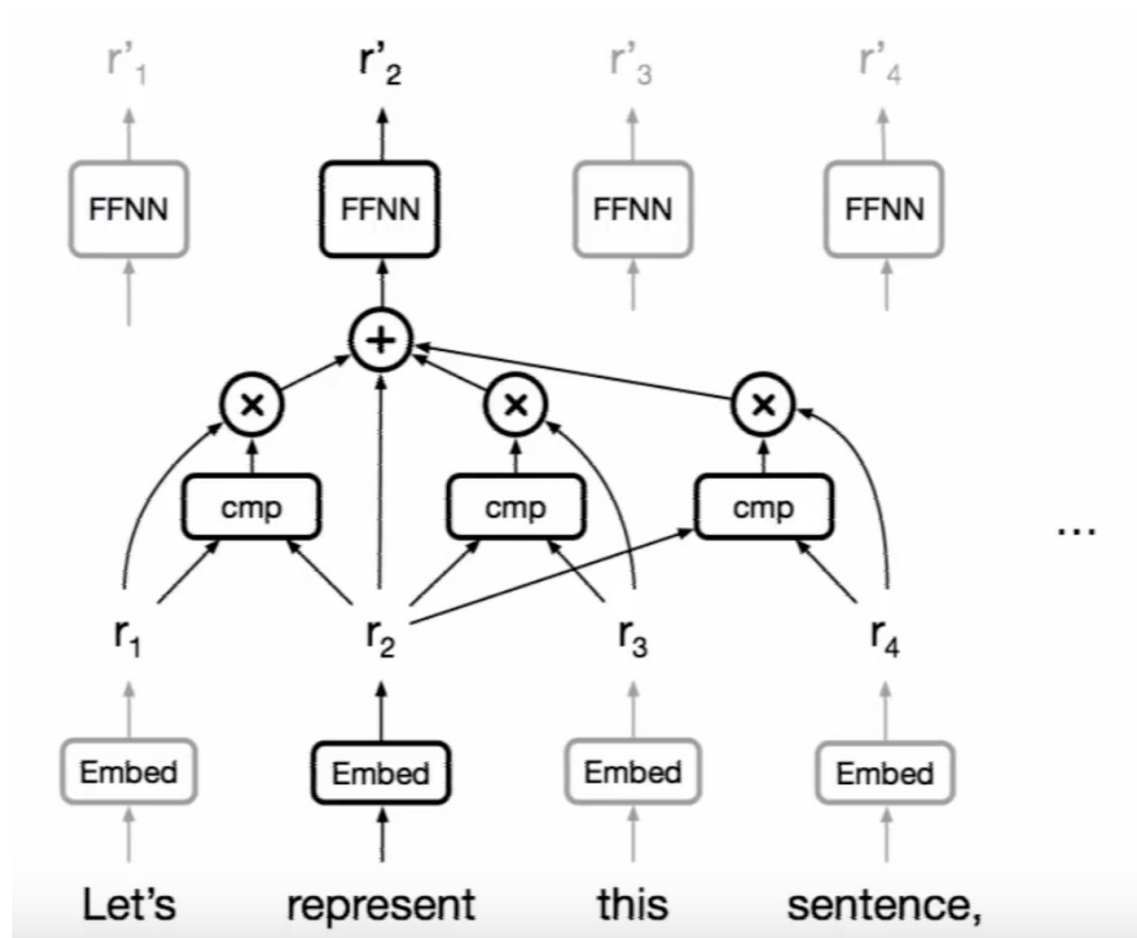
<https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b>

Self-Attention in Bert

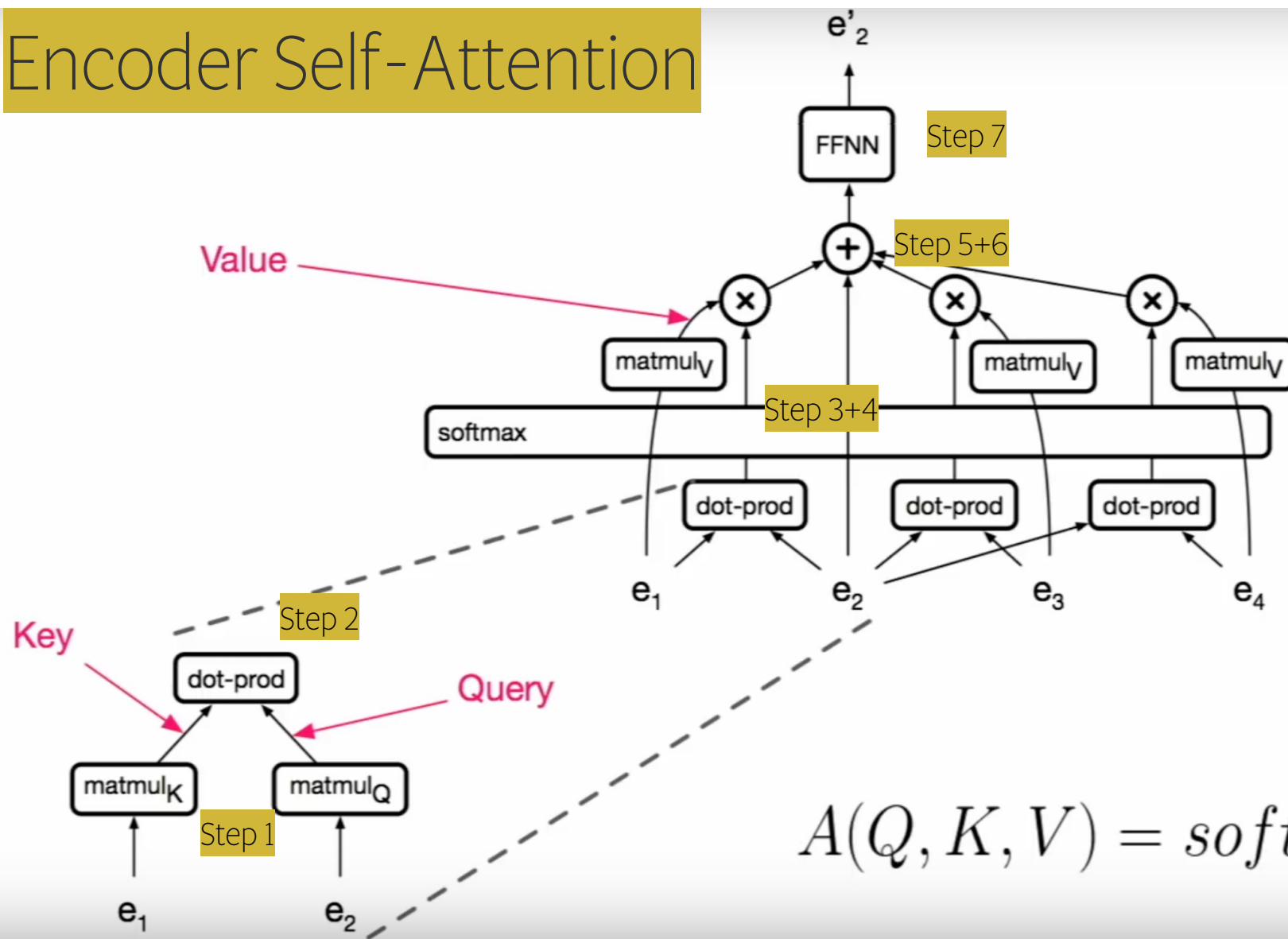
Step 7

Step 1-6

Step 0

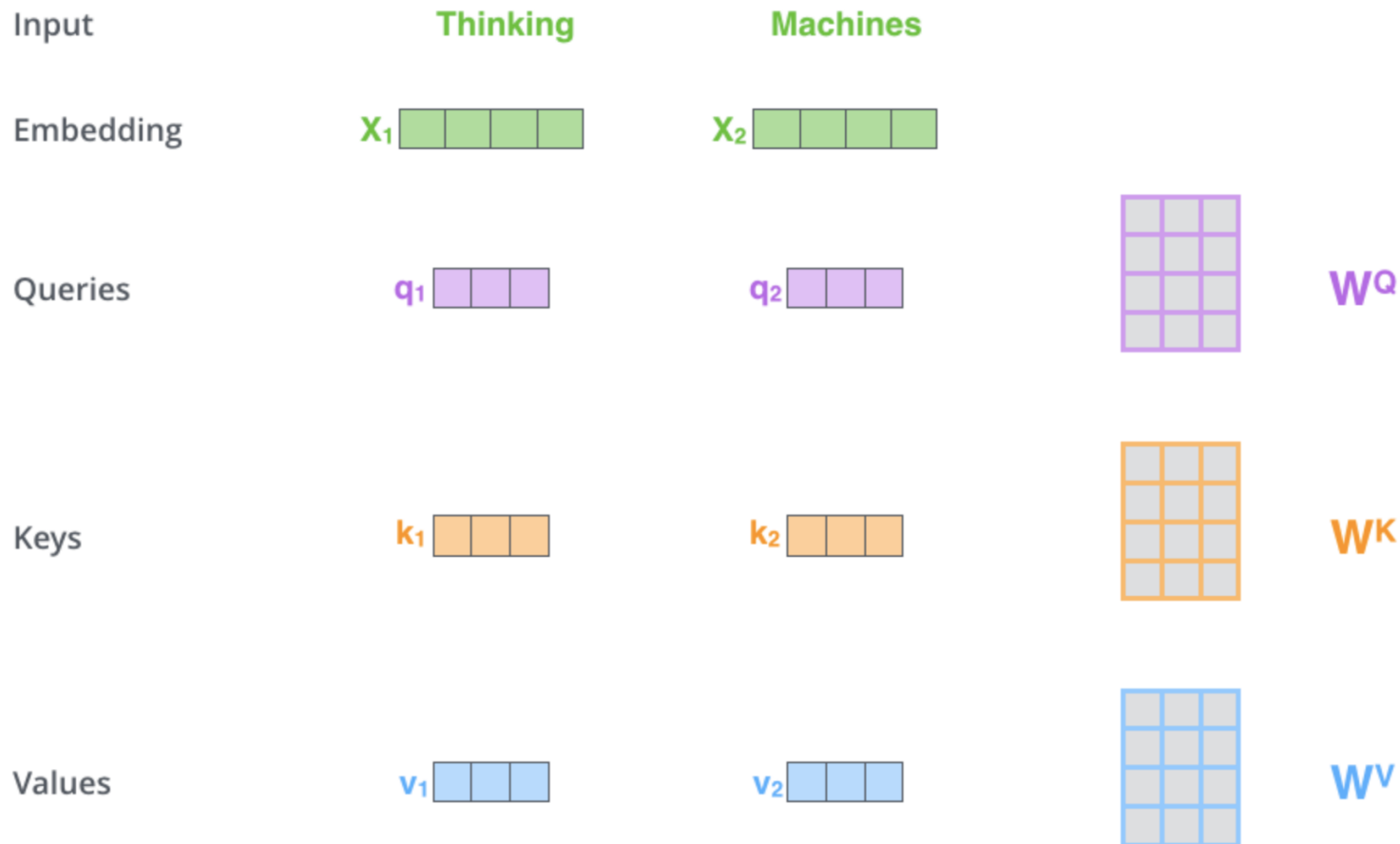


Encoder Self-Attention

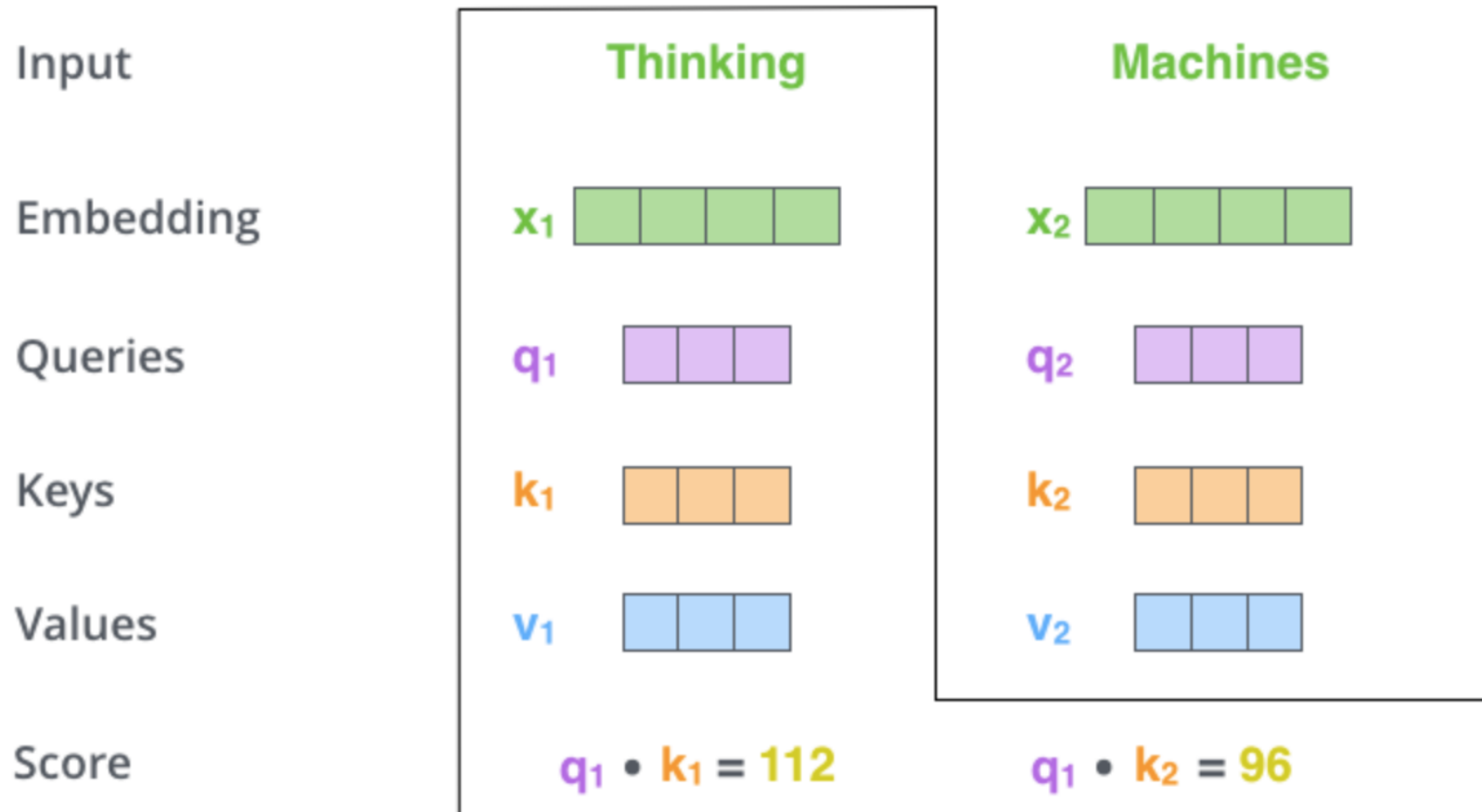


$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

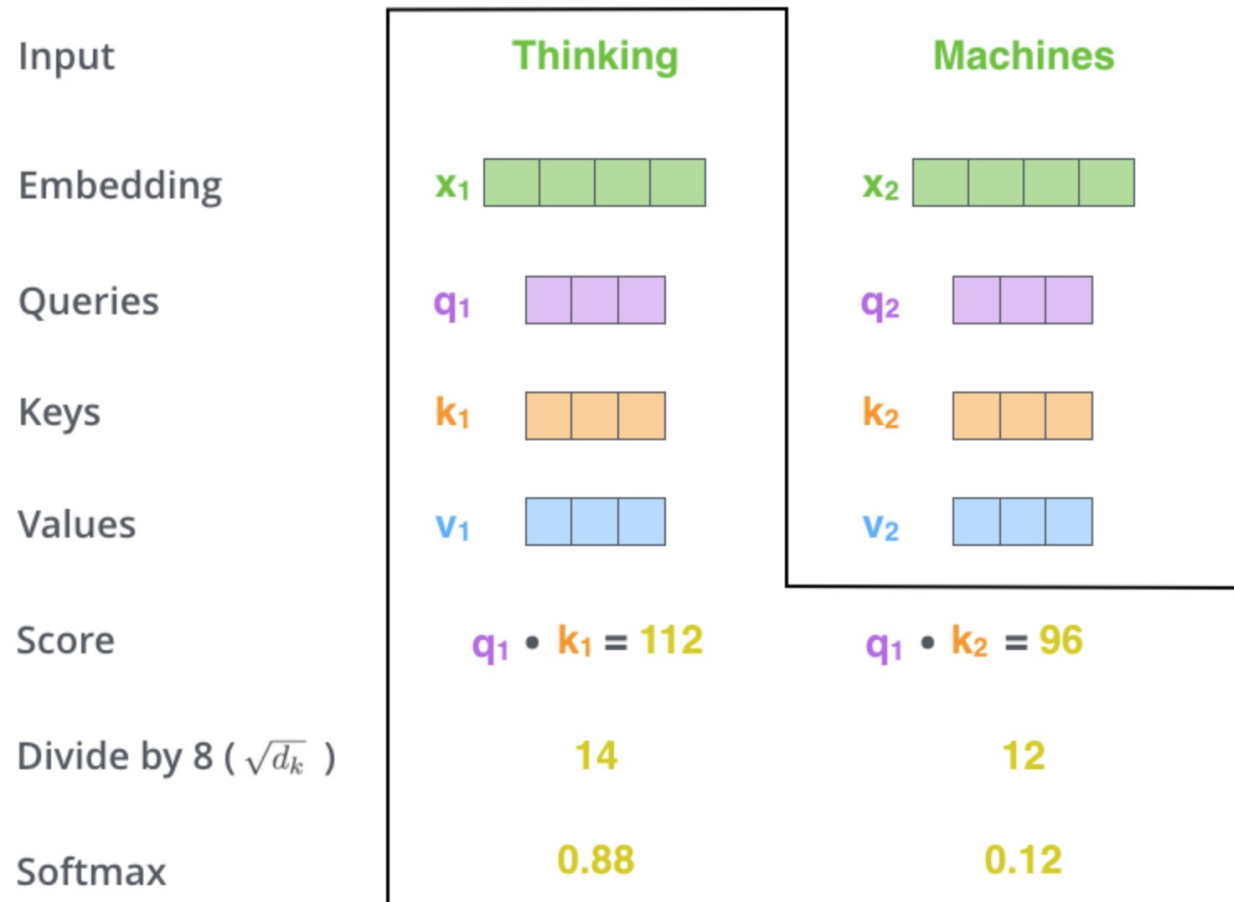
Step 1: Calculate vectors q, k, v <http://jalammar.github.io/illustrated-transformer/>



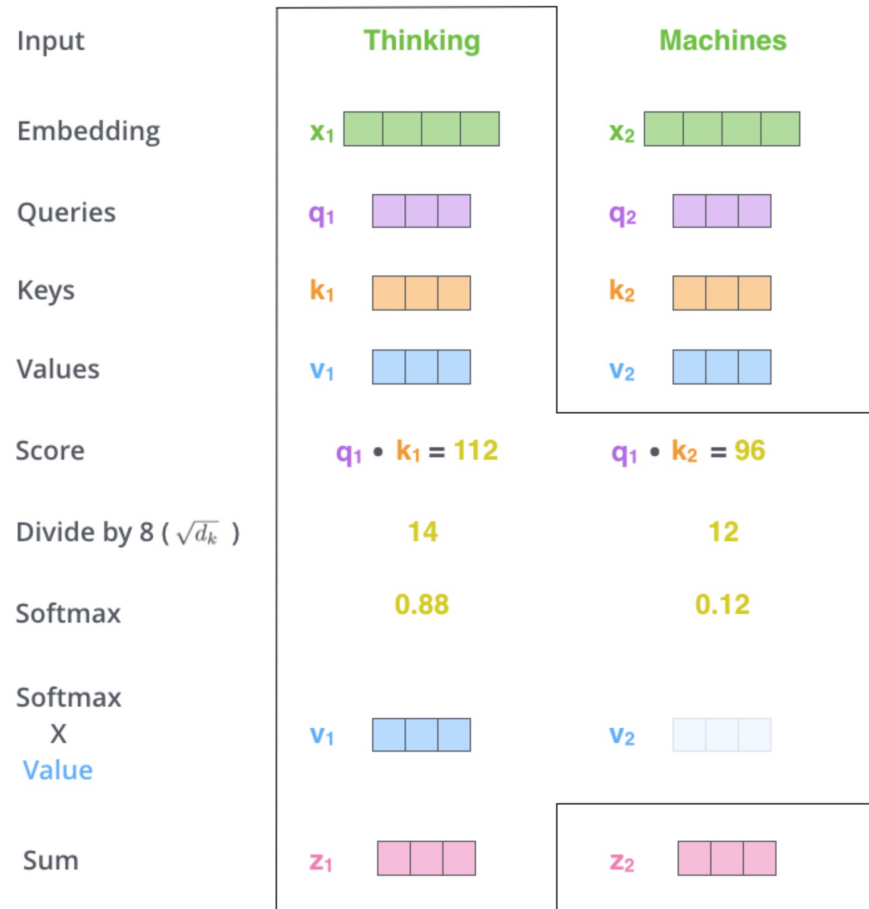
Step 2: Score each token pair <http://jalammar.github.io/illustrated-transformer/>



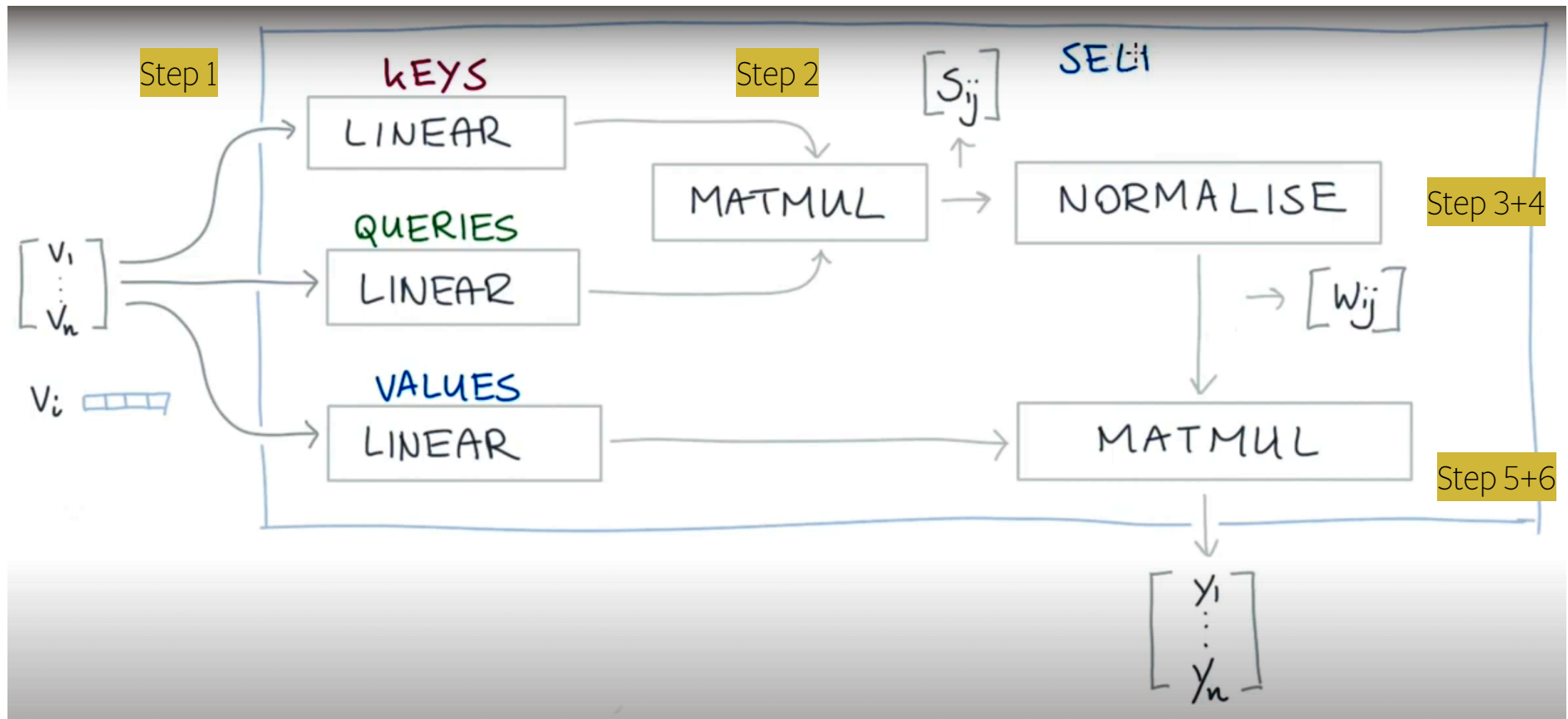
Step 3+4: Normalize+Softmax <http://jalammar.github.io/illustrated-transformer/>



Step 5+6: Weighted Sum <http://jalammar.github.io/illustrated-transformer/>

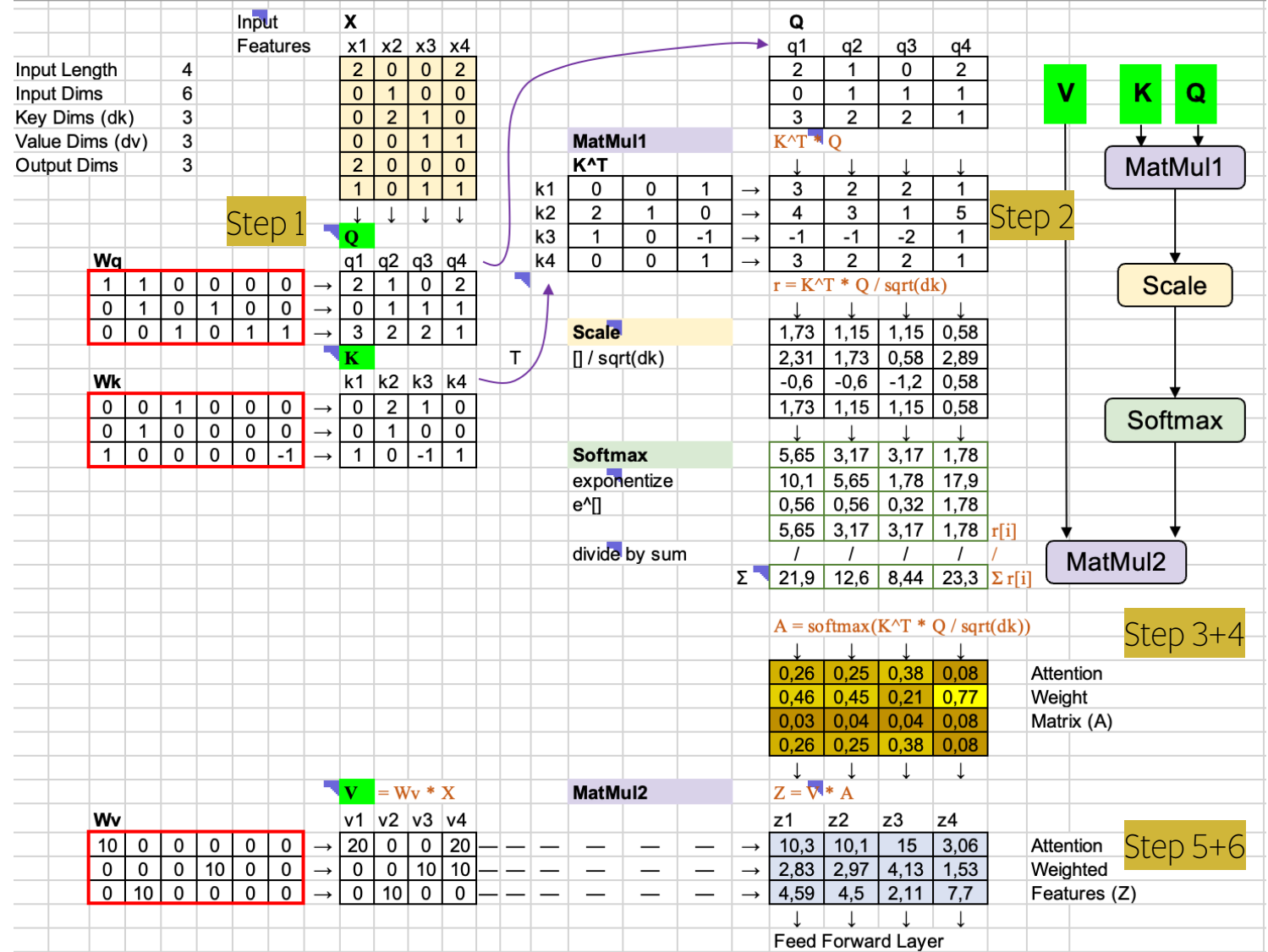


Overview attention module (<https://youtu.be/tlvKXrEDMhk?si=70BZ5TeJgG8JdXpw>)

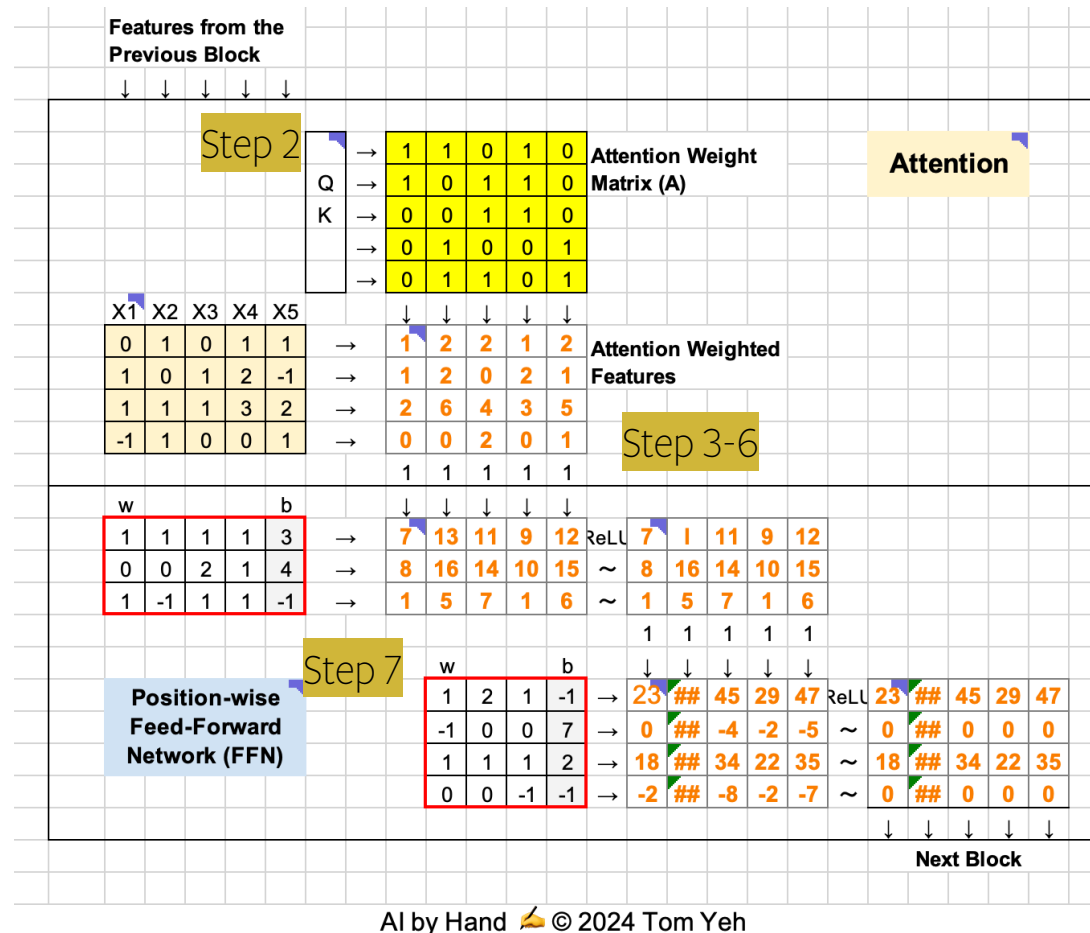


See “06-Self-Attention.xlsx”

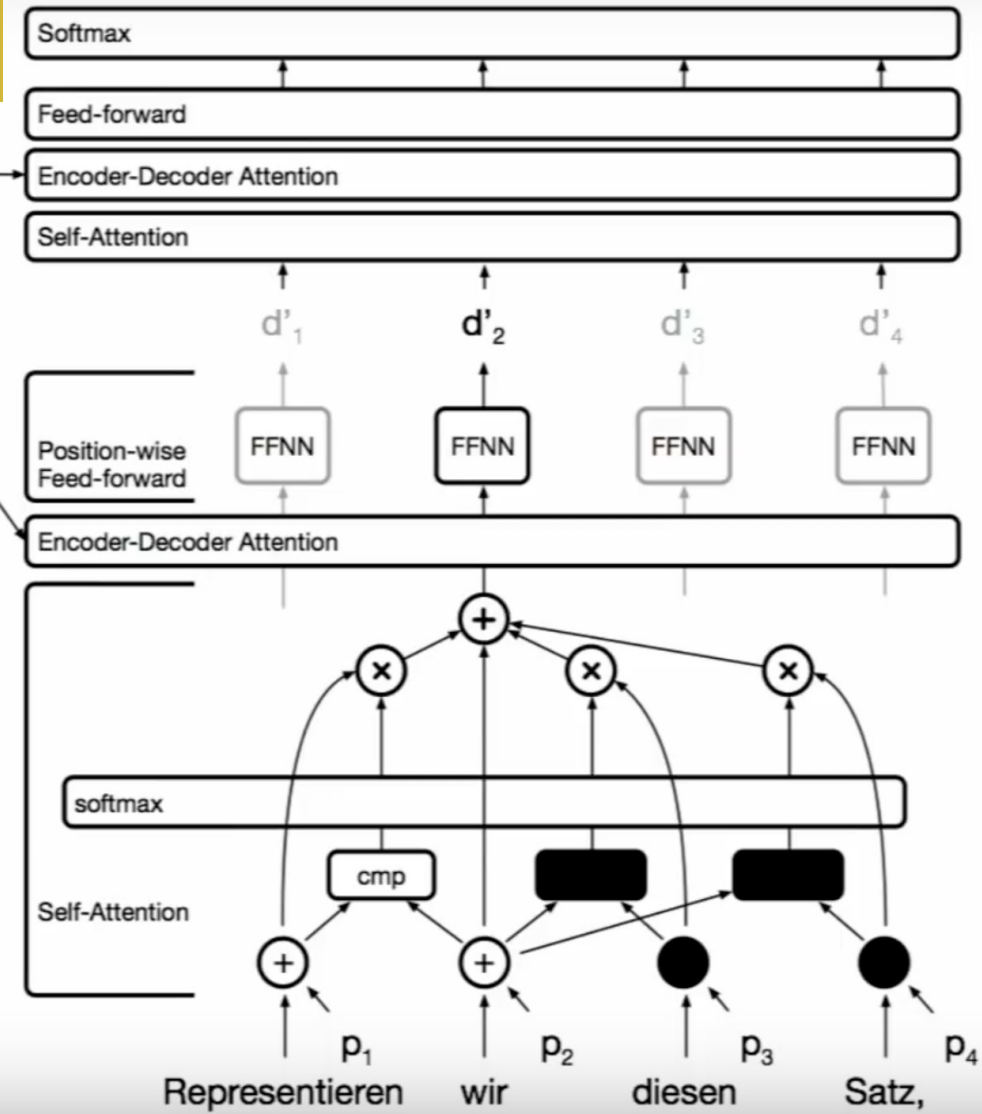
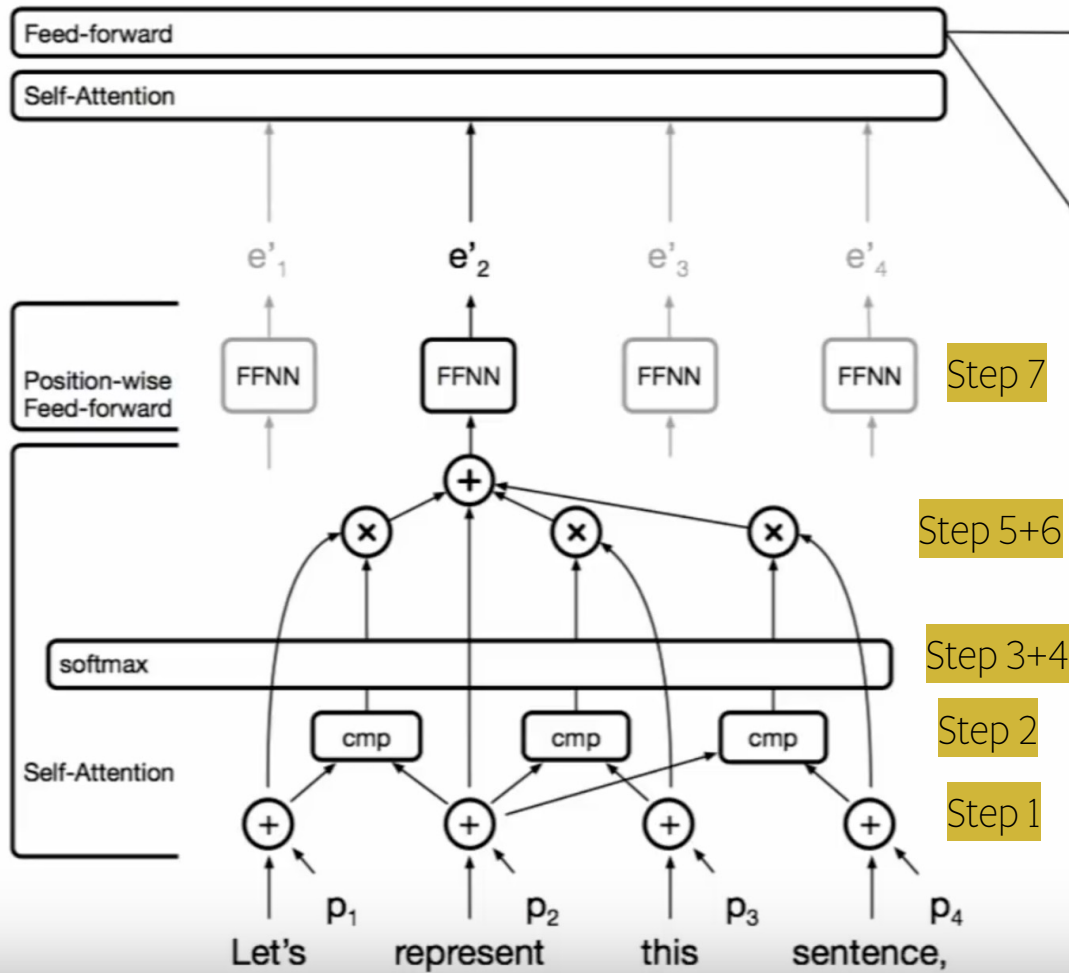
Self-Attention | AI by Hand 🙌 © 2024 Tom Yeh



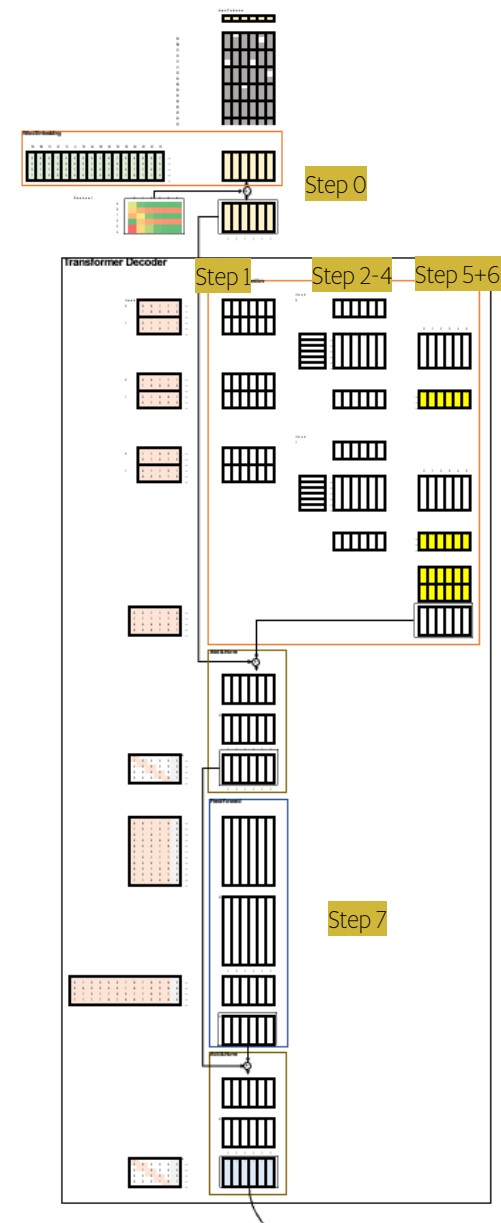
See “06-Transformer-simplifiedEncoderBlock.xlsx”

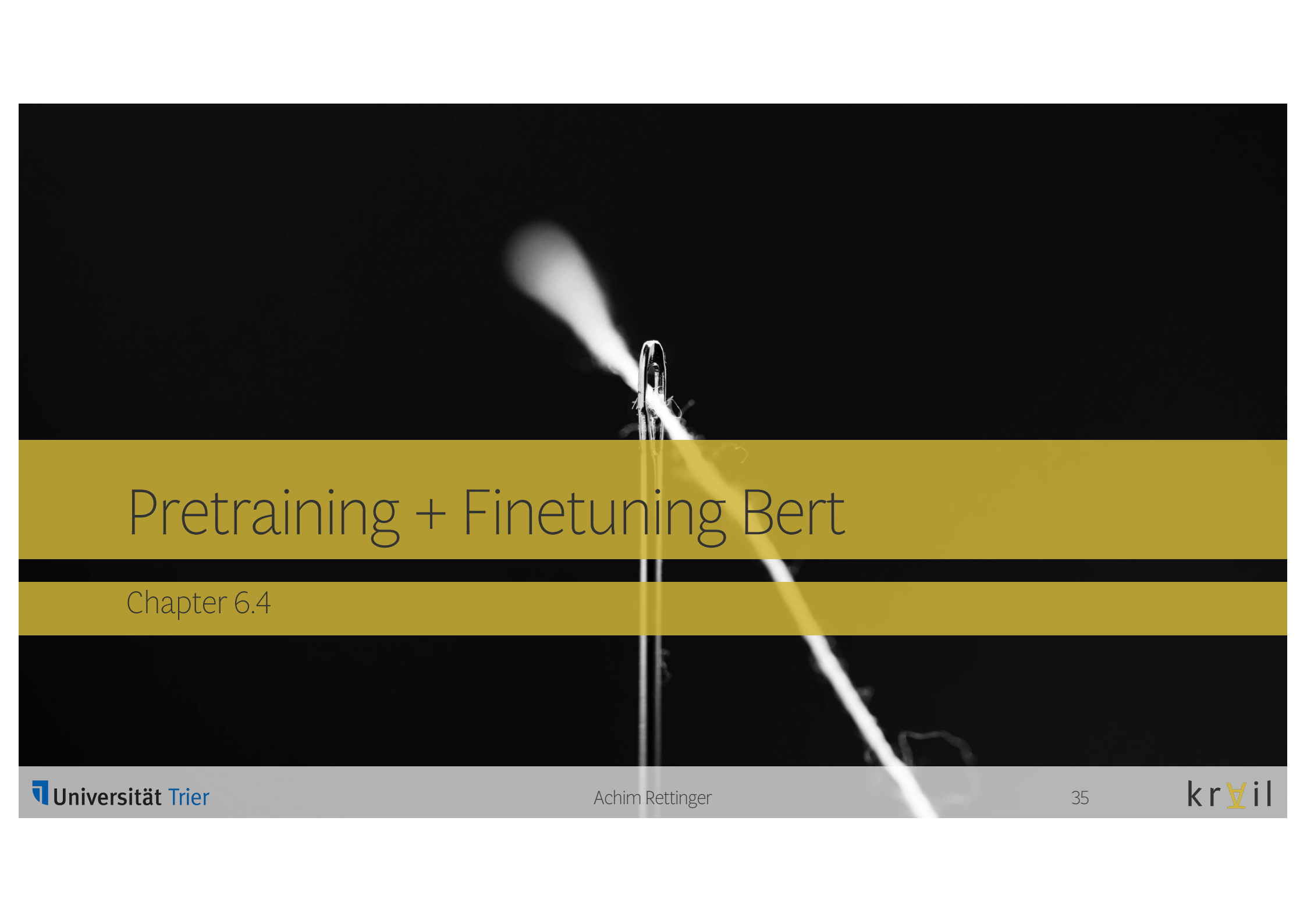


Encoder-Decoder Architecture



See “06-Transformer-
Full-Stack.xlsx”

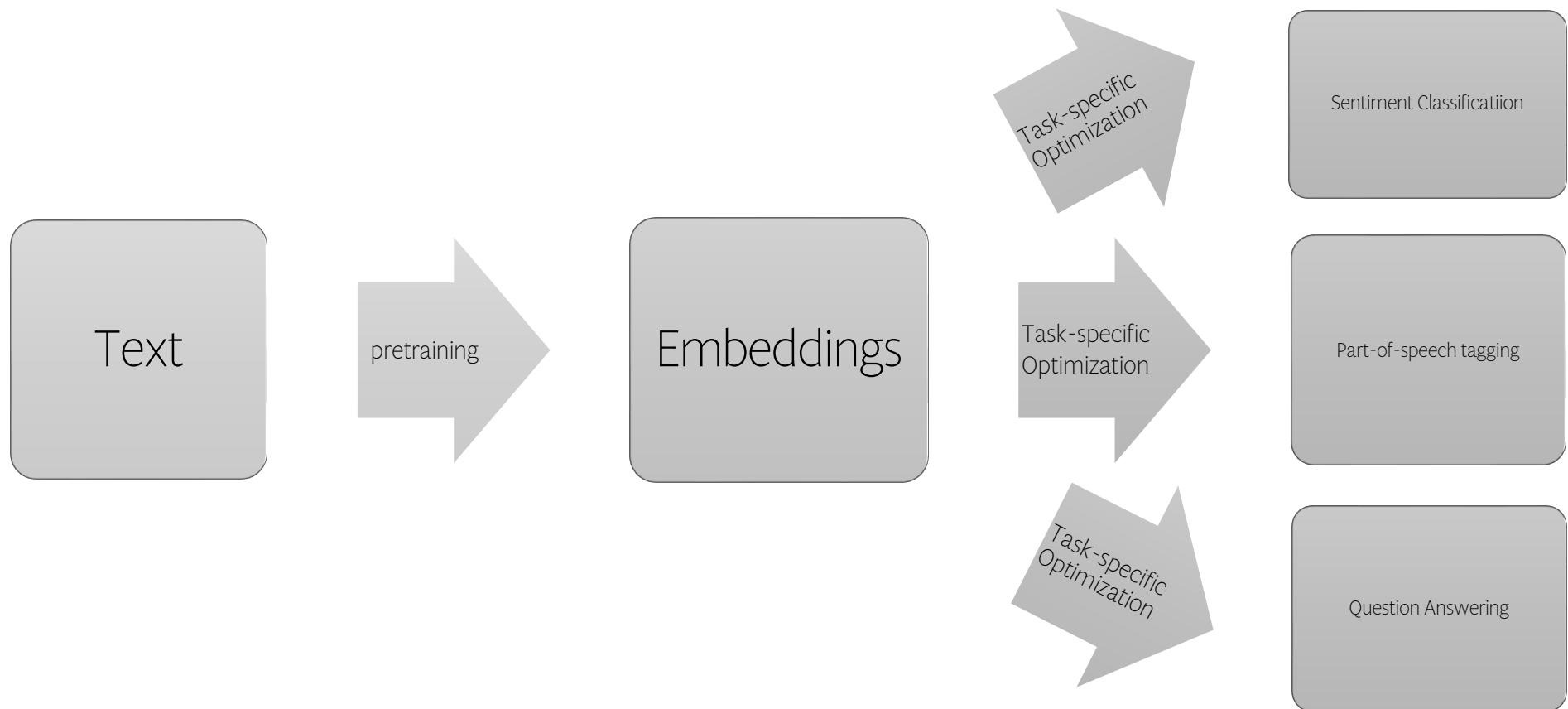




Pretraining + Finetuning Bert

Chapter 6.4

Text -> Pretraining -> Representation -> Finetuning -> Task



Self-supervised Pre-training

Task: Masked Language Modeling

- 15% of tokens are replaced by [MASK] :
- Objective: Predict what concrete token [MASK] replaces.

Task-specific Optimization (supervised)

- Sequence-Level:
 - Sentence Pair Classification
 - Single Sentence Classification
- Span-Level:
 - Question Answering
- Token-Level:
 - Single Sentence Tagging
- For each task a layer is added and (separately) trained

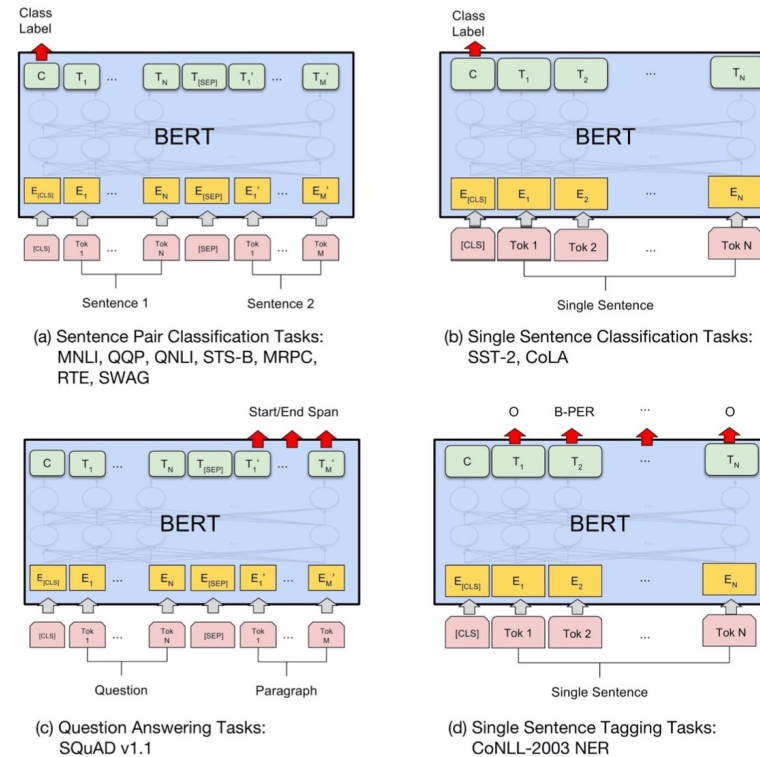


Figure 3: Our task specific models are formed by incorporating BERT with one additional output layer, so a minimal number of parameters need to be learned from scratch. Among the tasks, (a) and (b) are sequence-level tasks while (c) and (d) are token-level tasks. In the figure, E represents the input embedding, T_i represents the contextual representation of token i , [CLS] is the special symbol for classification output, and [SEP] is the special symbol to separate non-consecutive token sequences.

Towards: Transfer Learning

- If there isn't enough labeled training data to train your network from scratch for Task 2.
- And there already exists a network that is pre-trained on a similar task (e.g., Task 1)
- And, if Task 1 and Task 2 have the same input(/output) format.
- => Use the network (with minor adaptation) for Task 2

Zero-/One-/...-/Few-Shot Learning

- No / one / ... / few training data points available for Task 2

In-Context Learning

- Don't adapt any weights of the network during transfer

“The future of language is about the development of language models...”¹



1

Computerlinguistik | Universität Trier | cl.cldh.uni-trier.de